

Architectural Blueprint for a Zero-Cost AI-Driven Resume Builder

The modern recruitment landscape in 2025 and 2026 is characterized by a definitive shift from traditional document processing to structured data extraction and enrichment. As corporations increasingly rely on sophisticated Applicant Tracking Systems (ATS), the demand for machine-readable career data has made the legacy PDF format a bottleneck in the hiring pipeline. The design of a "Zero-Cost AI Resume Builder" addresses this friction by synthesizing generative artificial intelligence with open data standards to provide candidates with a high-fidelity, interoperable professional profile. This technical blueprint details the integration of the MERN (MongoDB, Express, React, Node.js) stack with the Gemini 1.5 Flash multimodal architecture and the GitHub REST API to deliver a production-grade application without recurring infrastructure costs.

Strategic Infrastructure and Hosting Paradigm

The foundational requirement for a zero-cost application is the strategic selection of service tiers that offer high functional ceilings while maintaining a \$0 monthly spend. The architecture utilizes a distributed hosting model to leverage the unique strengths of various Platform-as-a-Service (PaaS) and Database-as-a-Service (DBaaS) providers.

Frontend Hosting: Vercel Hobby Tier

For the client-side application, Vercel remains the industry standard for React-based single-page applications (SPAs). The Vercel Hobby tier is particularly advantageous for a resume builder due to its global Content Delivery Network (CDN) and seamless Git-driven deployment workflows. It provides 100 GB of bandwidth per month and automatic SSL termination via Let's Encrypt, ensuring that the user's career data is transmitted over secure channels. The frontend is architected as a lightweight interface that delegates heavy processing to the backend, maintaining rapid load times even on mobile devices.

Backend Hosting: Render vs. Koyeb Free Tiers

The selection of a backend environment for a MERN application in 2025 involves a critical trade-off between resource persistence and execution performance. While Render has long been a favorite for its simplicity, the emergence of Koyeb as a container-native platform offers superior performance for AI-intensive workloads.

Feature Category	Render Free Web Service	Koyeb Starter Plan
Compute Engine	Shared vCPU (0.1)	Firecracker MicroVM (0.1 vCPU)
Memory Allocation	512 MB RAM	512 MB RAM
Inactivity Policy	Spins down after 15m	Enforced Scale-to-Zero
Initial Latency	High (First request wait)	1-5s VM creation time
Bandwidth	100 GB / month	100 GB / month

Feature Category	Render Free Web Service	Koyeb Starter Plan
Egress Cost	\$0.10 per GB (after free)	\$0.04 per GB (after free)
Deployment	Git-based only	Git-based and Docker

The analysis suggests that Koyeb's use of Firecracker micro-VMs provides a more reliable execution environment for the multimodal processing required by the Gemini API. Koyeb's "Light Sleep" tier, scheduled for 2025, aims to bring wake-up times down to approximately 200ms, which would virtually eliminate the user experience friction currently associated with free-tier cold starts. However, Render's generous 750 free instance-hours per month remains a robust secondary option if the application requires more standard Node.js web server configurations.

Database Tier: MongoDB Atlas M0

Data persistence is managed through the MongoDB Atlas M0 cluster, which provides a globally distributed, multi-cloud NoSQL environment. The flexible schema of MongoDB is essential for a resume builder because resume data structures are inherently hierarchical and often subject to frequent schema updates as new fields are added.

The M0 cluster enforces specific limits that the architect must account for in the application logic:

- **Storage:** 512MB of data storage is provided. Given that a standard JSON Resume document is approximately 5-10 KB, this tier can support upwards of 50,000 unique resumes before requiring an upgrade.
- **Connections:** A hard limit of 500 concurrent connections is enforced. This necessitates efficient connection pooling within the Express.js application to prevent database lockouts during high-traffic events.
- **Data Transfer:** Combines ingress and egress limits of 10GB per rolling seven-day period. Exceeding this limit triggers network throttling rather than overage charges, which maintains the zero-cost requirement while preserving service availability.

AI Core: Gemini 1.5 Flash Multimodal Integration

The defining feature of this architecture is its integration with Google's Gemini 1.5 Flash API. This model is engineered specifically for low-latency, high-volume tasks such as document parsing and summarization.

Gemini 1.5 Flash Free Tier Dynamics

The Gemini API's free tier provides a unique competitive advantage for developers building resume tools. It offers a massive 1-million-token context window, allowing for the ingestion of extremely long profile documents and extensive repository READMEs without the need for complex Retrieval-Augmented Generation (RAG) pipelines.

Dimension	Specification (December 2025)	Strategy for 0 USD Spend
Requests Per Minute (RPM)	15 requests	Implement client-side debouncing and server-side queuing.
Tokens Per Minute (TPM)	1,000,000 tokens	Use standard PDF formatting to keep prompt sizes efficient.
Requests Per Day (RPD)	1,500 requests	Use local storage for draft

Dimension	Specification (December 2025)	Strategy for 0 USD Spend
		resumes to minimize API calls.
Context Window	1,048,576 tokens	Pass the entire LinkedIn PDF as a single multimodal part.
Privacy Policy	Data used for model training	Implement explicit user consent for PDF processing.

The December 2025 adjustments to the Gemini API quotas reduced the daily limit for Pro models while keeping the Flash model's daily limit at 1,500 RPD. This makes Gemini 1.5 Flash the primary engine for real-time parsing, while Pro models should be reserved for more complex analysis if the user upgrades to a paid tier.

Multimodal PDF Parsing Mechanism

Traditional resume parsing relies on simple text extraction libraries like pdf-parse, which often struggle with design-heavy resumes that use multi-column layouts. Gemini 1.5 Flash bypasses these limitations by using its native vision capabilities to "see" the document structure. When a user uploads a LinkedIn Profile PDF, the backend does not simply scrape the text; it sends the binary file to Gemini as an `InlineData` part. This allows the model to identify visual cues such as headers, subheaders, and bullet points, as well as the proximity of text blocks to one another. This approach ensures that the "Skills" section is correctly separated from "Experience" and that multi-column text is read in its intended logical order rather than simply from left-to-right across the page.

GitHub Synchronization and Portfolio Enrichment

To differentiate the generated resumes from standard static documents, the architecture integrates a real-time GitHub synchronization service. This service fetches the candidate's latest development work and uses AI to transform raw technical documentation into recruiter-friendly achievements.

The GitHub REST API Workflow

The application utilizes the GitHub REST API to retrieve repository information. To maximize the rate limit for end-users, the application should prompt users to provide a Personal Access Token (PAT) with `read:repo` scope.

- Repository Enumeration:** The backend calls `GET /users/{username}/repos` to retrieve a list of public repositories. The metadata includes the repository name, primary programming language, and star count.
- README Acquisition:** For repositories selected by the user, the application calls `GET /repos/{owner}/{repo}/readme`. The content of the `README.md` is returned as a Base64-encoded string, which the Node.js backend must decode into a standard UTF-8 string.
- Documentation Summarization:** The raw README text, which may contain thousands of lines of documentation or code snippets, is sent to Gemini 1.5 Flash. The model is prompted to "Summarize the core technical achievements and technologies used in this project into three concise bullet points for a professional resume".

This mechanism effectively automates the creation of a "Projects" section that reflects the user's

most current technical competencies, providing a significant advantage over manual resume updates.

Data Model: The JSON Resume Standard

The application adopts the JSON Resume standard (v1.0.0) as its primary data model. This open-source schema defines a normalized taxonomy for professional history, ensuring that the data remains portable across different themes, renderers, and ATS integrations.

JSON Schema Field Descriptions

The resume document is structured into several top-level objects, each with specific requirements for data integrity.

Section	Key Fields and Descriptions	Data Source Integration
basics	name, label, email, phone, summary, location	Extracted from LinkedIn introduction section.
work	company, position, startDate, endDate, highlights	Extracted from LinkedIn Experience section.
education	institution, area, studyType, score, courses	Extracted from LinkedIn Education section.
skills	name, level (e.g., Master, Expert), keywords	Mapping of LinkedIn Top Skills and endorsements.
projects	name, description, highlights, keywords, url	Enriched via GitHub README summarization.
awards	title, date, awarder, summary	Extracted from LinkedIn Accomplishments section.

The choice of JSON Resume allows for the separation of concerns between content and presentation. While the raw data is stored as JSON in MongoDB, the frontend can apply various CSS themes to render the data as a high-fidelity PDF or a responsive web portfolio.

API Contracts and Route Logic

The Express.js backend serves as the orchestrator for the AI and data sync operations. The following API contracts define the interaction between the client and the parsing engine.

POST /api/v1/resume/parse

This endpoint is responsible for taking a binary PDF file and returning a structured JSON Resume object. It utilizes multer with a memoryStorage configuration to handle file buffers securely.

Backend Logic Flow:

- Buffer Capture:** The multer middleware extracts the uploaded file and stores it in req.file.buffer.
- Multimodal Preparation:** The buffer is converted to a Base64 string for inclusion in the Gemini API payload.
- Generative AI Call:** The request is sent to the models.generateContent endpoint of Gemini 1.5 Flash. The configuration must include responseType: "application/json"

to enable Structured Output mode.

4. **Schema Validation:** Upon receiving the AI response, the backend validates the JSON against the JSON Resume specification using a library like ajv to ensure the data is safe for the frontend to render.
5. **Response:** The structured resume is returned to the user with a 200 OK status, or an appropriate error code if parsing fails.

GET /api/v1/github/sync/:username

This endpoint fetches repo names and descriptions to populate the resume projects section.

Backend Logic Flow:

1. **Request Parameters:** The username is extracted from the URL path. If a Personal Access Token is provided in the headers, it is used for authenticated requests.
2. **Axios Fetching:** An Axios instance calls the GitHub repos endpoint.
3. **Iterative Processing:** For repositories containing technical work, the README is fetched and summarized by Gemini.
4. **Data Merging:** The summarized project highlights are merged into the user's existing resume profile in MongoDB Atlas.

Prompt Engineering for Structural Integrity

To prevent AI hallucinations—such as the creation of non-existent job titles or the invention of graduation dates—the Gemini API requires a strictly defined system instruction. The system prompt guides the model's reasoning process and enforces adherence to the output schema.

The Exact System Prompt

The following text is configured as the `system_instruction` for the generative model:

You are a highly specialized Resume Extraction and Data Mapping Assistant. Your mission is to process the provided LinkedIn Profile PDF and map its content with 100% accuracy to the JSON Resume Schema (v1.0.0).

CRITICAL CONSTRAINTS:

1. **ZERO HALLUCINATION:** You are forbidden from inventing data. If a field such as "phone" or "email" is not present in the PDF, leave it as an empty string. If a whole section is missing, return an empty array.
2. **DATE STANDARDIZATION:** All dates must be returned in ISO 8601 format (YYYY-MM-DD). If the document only lists a year, default to YYYY-01-01. For ongoing roles, leave "endDate" as an empty string.
3. **HIERARCHICAL INTEGRITY:** Maintain the nesting of the schema exactly. Ensure "highlights" is an array of strings representing bullet points from the work description.
4. **SKILLS EXTRACTION:** Extract technical skills from the text and group them by appropriate categories in the "skills" array (e.g., "Web Technologies", "Programming Languages").
5. **OUTPUT:** Return ONLY the JSON object. Do not include any conversational text, markdown formatting, or explanations.

By explicitly stating these rules, the architect leverages the Gemini model's instruction-following

capabilities to ensure that the output is safe for programmatic consumption.

Monorepo Production File Tree

A clean, production-ready directory structure is essential for the maintainability of a full-stack JavaScript application. This monorepo uses a separation between the Vite-powered React client and the Node.js/Express server.

```
/zero-cost-resume-builder |—.env # Project secrets (API keys, DB URIs) |— package.json #
Root scripts for starting both projects |—.gitignore # Version control exclusions |— client/ #
Frontend Project (React + Vite) | |— src/ | | |— components/ # Reusable UI
(FileUpload.jsx, SyncButton.jsx) | | |— features/ # Feature modules (ResumePreview,
GitHubList) | | |— hooks/ # Custom state hooks (useResume.js) | | |— services/ # Axios
client wrappers (api.service.js) | | |— store/ # Global state (Zustand or Context) | | |—
App.jsx # Root UI component | |— index.html | |— package.json | |— vite.config.js |—
server/ # Backend Project (Express + Gemini) |— src/ | |— config/ # Gemini and MongoDB
setup (db.js, ai.js) | |— controllers/ # Route handlers (resume.controller.js) | |—
middleware/ # Multer and validation (upload.middleware.js) | |— models/ # Mongoose
Schemas (Resume.js, User.js) | |— routes/ # API Route definitions (api.v1.routes.js) | |—
services/ # Core logic (gemini.service.js, github.service.js) | |— app.js # Express app
initialization |— package.json |— server.js # Entry point for Render/Koyeb
```

This structure supports the "Service Layer" pattern, where controllers handle incoming requests and validation, while services interact with external APIs like Gemini and GitHub.

Step-by-Step Execution Roadmap

The implementation of the resume builder follows a six-phase plan, progressing from local development to a globally distributed, zero-cost production environment.

Phase 1: Environment and Core Scaffolding

1. **Initialize Monorepo:** Create the project root and subdirectories. Set up a root package.json that uses concurrently to run both client and server during development.
2. **API Provisioning:** Register for a Gemini API key via Google AI Studio and create a free M0 cluster on MongoDB Atlas.
3. **Security Setup:** Configure .env files with strict .gitignore rules to prevent API key exposure.

Phase 2: AI Parsing Logic Development

1. **Multer Integration:** Implement the upload.middleware.js using multer.memoryStorage() to ensure PDF files are never persisted to disk, reducing security risk.
2. **Gemini Service:** Develop the gemini.service.js module. Configure the multimodal request to send the PDF buffer as a Base64 string alongside the exact system prompt.
3. **Structured Output Implementation:** Configure the model to return JSON directly and test for consistency across various LinkedIn PDF formats.

Phase 3: GitHub Sync Implementation

- 1. **Octokit Configuration:** Integrate the `@octokit/rest` library for easier interaction with the GitHub API.
- 2. **README Summarization Pipeline:** Implement the iterative logic that fetches READMEs and prompts Gemini to convert technical details into resume highlights.
- 3. **Rate Limit Management:** Add logic to detect unauthenticated users and provide warnings about the 60-request-per-hour limit.

Phase 4: Frontend UI and State Management

- 1. **Vite Setup:** Initialize the frontend and install `tailwindcss` for professional, rapid styling.
- 2. **Async State Handling:** Use custom hooks to manage the asynchronous states of the resume parsing process (Idle, Loading, Success, Error).
- 3. **Resume Previewer:** Build a component that renders the JSON Resume data into a live-updating PDF preview using the `react-to-print` or `jspdf` libraries.

Phase 5: Persistence and Polish

- 1. **Mongoose Modeling:** Define the Resume schema to store historical user data and preferences.
- 2. **Error Handling:** Implement global error middleware on the Express server and toast notifications on the React client to handle API rate limits and invalid file formats.

Phase 6: Multi-Cloud Deployment

- 1. **Frontend (Vercel):** Connect the client/ directory to Vercel. Configure environment variables for the production API URL.
- 2. **Backend (Koyeb/Render):** Connect the server/ directory to the chosen provider. Ensure the health check route is active to minimize cold start duration.

Zero-Cost Compliance Audit

A successful implementation must avoid "hidden" costs such as outbound bandwidth overages or database storage fees. The following checklist ensures that the application remains free-to-operate.

Critical Service Tiers

Service	Tier Name	Free Limit Awareness	Verification ID
Hosting (Backend)	Koyeb Starter	512MB RAM / 0.1 vCPU	
Hosting (Frontend)	Vercel Hobby	100GB Bandwidth	
Database	Atlas M0	512MB / No backups	
AI Engine	Gemini Flash	1,500 Requests per	

Service	Tier Name	Free Limit Awareness	Verification ID
		day	
Repository API	GitHub REST	5,000 requests/hr (with token)	

Mandatory Library Stack

- **@google/generative-ai:** The primary SDK for Gemini 1.5 Flash. It is free to use and provides native support for multimodal document understanding.
- **multer:** Essential for handling PDF uploads without writing to disk. This preserves the ephemeral nature of the free hosting environment.
- **axios:** Used for all GitHub and server-client communication. It is lightweight and includes built-in support for request/response interceptors to handle 429 rate-limit errors.
- **cors:** Required to safely allow the Vercel-hosted frontend to communicate with the Render/Koyeb backend.
- **pdf-parse:** Used as a lightweight fallback for extracting text from extremely simple PDFs where vision-based inference would be overkill.

Architectural Challenges and Solutions

Building on free infrastructure requires specialized engineering patterns to circumvent service-level constraints.

Managing Cold Starts and Inactivity

The primary challenge of a zero-cost backend is the tendency for services to sleep after 15 minutes of inactivity. This results in a "bad user experience" where the first visitor must wait several seconds for the VM to wake up.

Solution: The application should implement a "System Waking" UI state. When the client app first loads on Vercel, it should immediately send a GET /health request to the backend. While the VM is initializing, the React app displays an informative message explaining that the "AI engine is warming up." This transparency increases user retention during the 1-5 second initialization phase.

Token Optimization and Prompt Performance

Gemini 1.5 Flash tokens for native PDF processing are calculated based on page count and resolution. Passing excessively large or blurry PDFs can lead to high token consumption and increased latency.

Solution: The frontend should perform client-side file compression and check that the PDF does not exceed the 50MB limit before attempting an upload. Furthermore, the system instructions should explicitly tell the model to ignore non-essential visual elements like profile headers or background graphics to focus the attention on semantic professional content.

GitHub API Rate Limit Strategies

Unauthenticated calls to the GitHub API are limited to 60 per hour, which can be exhausted quickly if multiple users are using the app from the same server IP.

Solution: The application *must* implement a token-based sync flow. By using the user's own PAT, the request budget is shifted from the server to the individual user, providing each candidate with 5,000 requests per hour—far more than is needed for a single resume session.

Conclusion and Strategic Recommendations

The "Zero-Cost AI Resume Builder" technical blueprint provides a roadmap for delivering a premium service with zero capital overhead. By leveraging the multimodal capabilities of Gemini 1.5 Flash, the application achieves a level of parsing accuracy previously unattainable with standard Node.js libraries. The adoption of the JSON Resume standard ensures that candidates own their data in a portable, machine-readable format that is future-proof against changes in the recruitment industry.

Key strategic takeaways for the architect include:

- **Prioritize Flash over Pro:** For real-time document parsing, the 15 RPM and 1,500 RPD limits of Gemini 1.5 Flash are superior for a free-tier user experience compared to the more restrictive Pro limits.
- **Firecracker Advantage:** Koyeb's micro-VM infrastructure is better suited for intermittent AI workloads than Render's standard web services, especially given the upcoming improvements to scale-to-zero wake times.
- **Enforcement of Schema:** Using Structured Output mode is not optional; it is the only way to guarantee a hallucination-free experience that integrates reliably with a MERN backend.

This architectural approach allows for rapid prototyping and deployment, providing a scalable foundation that can easily transition to paid tiers should the application experience significant viral growth. By focusing on the nuances of free-tier quotas and multimodal AI integration, the developer can provide a high-value tool that bridges the gap between traditional resumes and the data-driven recruitment ecosystem of tomorrow.

Works cited

1. Schema & Structure - JSON Resume, <https://docs.jsonresume.org/schema> 2. JSON Resume: Standardized CV Format for Developers and Automation - Angelo Lima, <https://angelo-lima.fr/en/json-resume-standardized-cv-format-developers-automation/> 3. The Complete Guide to MERN Stack Development in 2025 - Tuvoc, <https://www.tuvoc.com/blog/complete-guide-mern-stack-development/> 4. Best free hosting platforms in 2026 - Appwrite, <https://appwrite.io/blog/post/free-hosting-platform> 5. Looking for Free Hosting Options for a Full MERN Stack App · community · Discussion #175658 - GitHub, <https://github.com/orgs/community/discussions/175658> 6. Best Hosting Platforms for MERN Projects - GeeksforGeeks, <https://www.geeksforgeeks.org/blogs/best-hosting-platforms-for-mern-projects/> 7. Render vs Fly.io, <https://render.com/articles/render-vs-fly-io> 8. Render vs Koyeb, <https://www.koyeb.com/docs/compare/render-vs-koyeb> 9. Compare Koyeb vs. Render - G2, <https://www.g2.com/compare/koyeb-vs-render-render> 10. Zero-Cost SaaS: How I Deployed My Project with Koyeb.com | by Mehmet Cevheri Bozoğlu, <https://medium.com/@cevheri/zero-cost-saas-how-i-deployed-my-project-with-koyeb-com-90518853a2ea> 11. Deploy AI Infrastructure in 2025: Serverless GPUs, Autoscaling, Scale-to-Zero, and More!

<https://www.koyeb.com/blog/deploy-ai-infrastructure-in-2025-autoscaling-serverless-gpus-scale-to-zero-and-more> 12. Deploy a MERN Full Stack Website for Free : full guide - DEV Community, <https://dev.to/suheldevs/deploy-a-mern-full-stack-website-for-free-full-guide-94o> 13. MERN Stack for Mobile App Development: A Complete Guide for 2025, <https://www.aalpha.net/blog/mern-stack-for-mobile-app-development/> 14. Atlas M0 (Free Cluster) Limits - Atlas - MongoDB Docs, <https://www.mongodb.com/docs/atlas/reference/free-shared-limitations/> 15. Does MongoDB Atlas Free Tier have any data transfer limits like Firestore? - Reddit, https://www.reddit.com/r/mongodb/comments/1lqjiik/does_mongodb_atlas_free_tier_have_any_data/ 16. Request limit in the free tier - MongoDB Atlas, <https://www.mongodb.com/community/forums/t/request-limit-in-the-free-tier/9804> 17. Atlas Service Limits - Atlas - MongoDB Docs, <https://www.mongodb.com/docs/atlas/reference/atlas-limits/> 18. Everything you need to know about the Gemini API as a developer in less than 5 minutes, <https://medium.com/around-the-prompt/everything-you-need-to-know-about-the-gemini-api-as-a-developer-in-less-than-5-minutes-5e75343ccff9> 19. Updated production-ready Gemini models, reduced 1.5 Pro pricing, increased rate limits, and more - Google Developers Blog, <https://developers.googleblog.com/en/updated-gemini-models-reduced-15-pro-pricing-increased-rate-limits-and-more/> 20. Learn about supported models | Firebase AI Logic - Google, <https://firebase.google.com/docs/ai-logic/models> 21. Gemini API Free Tier Limits 2025: Complete Guide to Rate Limits, 429 Errors & Solutions, <https://aifreeapi.com/en/posts/gemini-api-free-tier-limit> 22. Rate limits - Google Gemini API, <https://gemini-api.apidog.io/doc-965865> 23. Google Gemini API Pricing 2026: Complete Cost Guide per 1M Tokens - MetaCTO, <https://www.metactio.com/blogs/the-true-cost-of-google-gemini-a-guide-to-api-pricing-and-integration> 24. Gemini Advanced Rate Limit: Complete 2025 Guide with December Updates, <https://aifreeapi.com/en/posts/gemini-advanced-rate-limit> 25. 7 PDF Parsing Libraries for Extracting Data in Node.js - Strapi, <https://strapi.io/blog/7-best-javascript-pdf-parsing-libraries-nodejs-2025> 26. Optimizing PDF Resume Files for Faster Parsing on Job Boards, <https://www.resumly.ai/blog/optimizing-pdf-resume-files-for-faster-parsing-on-job-boards> 27. Document understanding | Gemini API - Google AI for Developers, <https://ai.google.dev/gemini-api/docs/document-processing> 28. Does PDF fine-tuning focus solely on text extraction, or does it also perform visual inference? - Gemini API - Google AI Developers Forum, <https://discuss.ai.google.dev/t/does-pdf-fine-tuning-focus-solely-on-text-extraction-or-does-it-also-perform-visual-inference/70280> 29. File input methods | Gemini API | Google AI for Developers, <https://ai.google.dev/gemini-api/docs/file-input-methods> 30. What is CV Parsing? How AI Reads Resumes for You - Adeptiq - Simply the easiest ATS, <https://adeptiq.be/blog/what-is-cv-parsing-how-ai-reads-resumes-for-you> 31. Getting started with the REST API - GitHub Docs, <https://docs.github.com/rest/using-the-rest-api/getting-started-with-the-rest-api> 32. REST API endpoints for repository contents - GitHub Docs, <https://docs.github.com/rest/repos/contents> 33. Github API List all repositories and repo's content - Stack Overflow, <https://stackoverflow.com/questions/14390090/github-api-list-all-repositories-and-repos-content> 34. API Fetch of GitHub Repositories with ReactJS | by Kehinde | Medium, <https://medium.com/@balogunkehinde3/api-fetch-of-github-repositories-with-reactjs-2eb9a21e80bd> 35. Contents | GitHub API - LFE Documentation, <https://docs2.lfe.io/v3/repos/contents/> 36.

How work with github api to retrieve a repository's readme file? - Stack Overflow, <https://stackoverflow.com/questions/75538091/how-work-with-github-api-to-retrieve-a-repository-s-readme-file> 37. How to Process PDFs with Gemini AI and JavaScript: A Beginner's Guide | by Amy Abafor, <https://medium.com/@amyabafor013/how-to-process-pdfs-with-gemini-ai-and-javascript-a-beginners-guide-13640a16a554> 38. JSON Resume Documentation, <https://docs.jsonresume.org/> 39. Modern CV Technology: JSON Resume embedded in HTML - Paul Hammant's blog, <https://paulhammant.com/2025/10/12/modern-cv-tech-json-resume-schema/> 40. mydnicq/rest-api-example: An example of a REST API in node.js. - GitHub, <https://github.com/mydnicq/rest-api-example> 41. MERN Best Practices 2025. Master modern patterns and industry... | by Sivanathan Dilakshan | Medium, <https://medium.com/@sivanathandilakshan/mern-best-practices-2025-9d2d0ad9cd1b> 42. multer - npm, <https://www.npmjs.com/package/multer> 43. How to Build File Upload APIs with Express and Multer - OneUptime, <https://oneuptime.com/blog/post/2026-01-27-express-multer-file-upload/view> 44. node.js - how to upload and read a file with express - Stack Overflow, <https://stackoverflow.com/questions/52467544/how-to-upload-and-read-a-file-with-express> 45. Structured outputs | Gemini API - Google AI for Developers, <https://ai.google.dev/gemini-api/docs/structured-output> 46. How to Use Gemini Structured Output and JSON Mode for Reliable Data Extraction, <https://oneuptime.com/blog/post/2026-02-17-how-to-use-gemini-structured-output-and-json-mode-for-reliable-data-extraction/view> 47. Improving Structured Outputs in the Gemini API - Google Blog, <https://blog.google/innovation-and-ai/technology/developers-tools/gemini-api-structured-outputs/> 48. JSON Resume Schema, <https://docs.rxresu.me/guides/json-resume-schema> 49. File Upload With Multer in Node.js and Express | Envato Tuts+, <https://code.tutsplus.com/file-upload-with-multer-in-node--cms-32088t> 50. Search Repositories by Webpage · community · Discussion #167920 - GitHub, <https://github.com/orgs/community/discussions/167920> 51. Scripting with the REST API and JavaScript - GitHub Docs, <https://docs.github.com/rest/guides/scripting-with-the-rest-api-and-javascript> 52. Structured output | Generative AI on Vertex AI - Google Cloud Documentation, <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/control-generated-output> 53. Generate structured output (like JSON and enums) using the Gemini API | Firebase AI Logic, <https://firebase.google.com/docs/ai-logic/generate-structured-output> 54. Best Practices for Structuring MEAN/MERN/MEVN Projects | Tutorials - Docs, <https://docs.codecapsules.io/tutorials/best-practices-for-structuring-mean-mern-mevn-projects> 55. Best practices for structuring MERN stack project with separate client and server directories, <https://community.webshinetech.com/t/best-practices-for-structuring-mern-stack-project-with-separate-client-and-server-directories/3653> 56. Recommended Folder Structure for React 2025 - DEV Community, https://dev.to/pramod_boda/recommended-folder-structure-for-react-2025-48mc 57. Popular Libraries 2025 for PDF Generation Using Node JS - pdf noodle, <https://pdfnoodle.com/blog/popular-libraries-2025-for-pdf-generation-using-node-js> 58. 7 Stunning JavaScript PDF Library 2025 - ThemeSelection, <https://themeselection.com/javascript-pdf-library/> 59. AI-Powered Plant Analysis Tool — Built with Node.js, Google Gemini, and PDFKit - Medium, <https://medium.com/@imavinashverma99/ai-powered-plant-analysis-tool-built-with-node-js-goog>

le-gemini-and-pdfkit-005ec9fbec77 60. Koyeb vs Render - GetDeploying,
<https://getdeploying.com/koyeb-vs-render> 61. Gemini Developer API pricing,
<https://ai.google.dev/gemini-api/docs/pricing> 62. axios/axios: Promise based HTTP client for the
browser and node.js - GitHub, <https://github.com/axios/axios> 63. Node.js PDF Library
Comparison (Free & Paid Tools) - IronPDF,
<https://ironpdf.com/nodejs/blog/compare-to-other-components/node-pdf-library/> 64. Heroku vs
Render: An Honest Comparison for 2026 - ExpressTech,
<https://expresstech.io/heroku-vs-render-an-honest-comparison-for-2026/> 65. Koyeb Alternatives
That Developers Actually Prefer in 2026, <https://kuberns.com/blogs/post/koyeb-alternatives/> 66.
Document understanding | Generative AI on Vertex AI - Google Cloud Documentation,
<https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/document-understanding>
67. Analyze documents (like PDFs) using the Gemini API | Firebase AI Logic - Google,
<https://firebase.google.com/docs/ai-logic/analyze-documents>